

Deep Neural Networks - Mid-Semester 1 Shorts

Contents

Session 1 - Neural-network fundamentals	5
Intuition	5
Definitions and jargon	5
Formulas, symbols, shapes, and assumptions	5
Conditional properties	5
Important distinctions	5
Checked example	5
Traps and counterexamples	5
Answer-free recall	5
Session 2 - Multilayer perceptrons	6
Intuition	6
Definitions and jargon	6
Formulas, symbols, shapes, and assumptions	6
Conditional properties	6
Important distinctions	6
Checked example	6
Traps and counterexamples	6
Answer-free recall	6
Session 3 - Forward pass, backpropagation, and outputs	7
Intuition	7
Definitions and jargon	7
Formulas, symbols, shapes, and assumptions	7
Conditional properties	7
Important distinctions	7
Checked example	7
Traps and counterexamples	7
Answer-free recall	8
Session 4 - Optimizing deep models	8
Intuition	8
Definitions and jargon	8
Formulas, symbols, shapes, and assumptions	8
Conditional properties	8
Important distinctions	8
Checked example	8

Traps and counterexamples	8
Answer-free recall	9
Session 5 - Generalization and stable training	9
Intuition	9
Definitions and jargon	9
Formulas, symbols, shapes, and assumptions	9
Conditional properties	9
Important distinctions	9
Checked example	9
Traps and counterexamples	9
Answer-free recall	10
Session 6 - Convolutional networks	10
Intuition	10
Definitions and jargon	10
Formulas, symbols, shapes, and assumptions	10
Conditional properties	10
Important distinctions	10
Checked example	10
Traps and counterexamples	10
Answer-free recall	11
Session 7 - CNN design and transfer learning	11
Intuition	11
Definitions and jargon	11
Formulas, symbols, shapes, and assumptions	11
Conditional properties	11
Important distinctions	11
Checked example	11
Traps and counterexamples	11
Answer-free recall	11
Session 8 - Integrated diagnosis and review	12
Intuition	12
Definitions and jargon	12
Formulas, symbols, shapes, and assumptions	12
Conditional properties	12
Important distinctions	12
Checked example	12

Traps and counterexamples	12
Answer-free recall	13

Session 1 - Neural-network fundamentals

Intuition

AI covers machines performing intelligent-seeming tasks. Machine learning (ML) learns patterns from data; deep learning (DL) is ML with multilayer neural networks. Supervised data have labels, unsupervised data do not, semi-supervised learning mixes both, and reinforcement learning uses actions and rewards.

Definitions and jargon

A feature is an input and a label the desired output. A weight scales a feature, a bias shifts the boundary, an activation transforms the score, and a loss measures quality. In a simplified analogy, dendrites receive signals, the axon hillock integrates, and the axon carries output; an artificial neuron is only mathematics. Connectionism builds behaviour from connected simple units.

Formulas, symbols, shapes, and assumptions

For length- d feature and weight columns x, w , $z = w^T x + b$; T turns a column into a row, so $w^T x$ sums paired products. Prediction $\hat{y} = f(z)$ is 1 when $z \geq 0$, else 0. Update $w \leftarrow w + \eta(y - \hat{y})x$, $b \leftarrow b + \eta(y - \hat{y})$, with learning rate $\eta > 0$. The error $y - \hat{y}$ is not a universal loss.

Conditional properties

A perceptron's boundary $w^T x + b = 0$ is linear although its step is nonlinear. Linearly separable means one line (or flat higher-dimensional boundary) separates the classes. Convergence needs finite, linearly separable data and repeated standard updates; XOR has no such promise.

Important distinctions

Label availability separates supervised, unsupervised, and semi-supervised learning; reinforcement learning uses action-reward interaction. Bias makes the score map affine, not linear. Training adjusts parameters; inference uses them.

Checked example

For AND with $w = (1, 1)$, the positive row $(1, 1)$ needs $2 + b \geq 0$, while $(1, 0)$ and $(0, 1)$ need $1 + b < 0$. Hence exactly $-2 \leq b < -1$; choose $b = -1.4$. The four scores are $-1.4, -0.4, -0.4, 0.6$, giving $0, 0, 0, 1$.

Traps and counterexamples

XOR's positive points are opposite corners, so one line cannot separate them. Not every AI uses ML or every DL task labels; bias is not linear, and artificial neurons are not biological equivalents.

Answer-free recall

Why does the threshold convention change the AND bias interval? Perform one perceptron update. Why does the nonlinear step leave one perceptron's boundary linear? [Know More](#) | [Detailed](#) | [External Sources](#)

Session 2 - Multilayer perceptrons

Intuition

One perceptron draws one boundary. An MLP composes many units so early layers create useful intermediate features and later layers combine them. Nonlinear activations prevent the entire stack from collapsing into one affine calculation.

Definitions and jargon

The input layer holds features, a hidden layer makes internal representations, and the output layer makes the prediction. Width is units in a layer; depth is successive computational transformations. State whether the non-computing input placeholder is counted. A dense layer connects every input to every unit. An affine map is a weighted linear map plus a bias.

Formulas, symbols, shapes, and assumptions

For a dense layer with input column x of shape $n \times 1$, weights W of shape $m \times n$, bias b of shape $m \times 1$, $z = Wx + b$ and $h = \phi(z)$, both $m \times 1$; ϕ is the activation. It has $mn + m$ trainable parameters. A $3 \rightarrow 5 \rightarrow 2$ MLP has $3 \cdot 5 + 5 + 5 \cdot 2 + 2 = 32$ parameters.

Conditional properties

With suitable nonlinear activation and enough units, universal-approximation results establish existence of parameters approximating certain functions, such as continuous functions on compact domains. They do not promise a small network, successful optimization, good test performance, or a required hidden-layer count.

Important distinctions

Depth and width are different. Representation capacity is different from trainability and generalization. Without nonlinear activations, composing $W_2(W_1x + b_1) + b_2$ gives one affine map $(W_2W_1)x + (W_2b_1 + b_2)$.

Checked example

Build XOR with two hidden threshold units: $h_1 = \text{OR}(x_1, x_2)$ and $h_2 = \text{NAND}(x_1, x_2)$, then output $\text{AND}(h_1, h_2)$. For $(1, 1)$, hidden values are $(1, 0)$ and output is 0; for $(1, 0)$, they are $(1, 1)$ and output is 1.

Traps and counterexamples

Do not infer that deeper is always better, that every definition of DNN counts hidden layers identically, or that the universal-approximation theorem explains how gradient descent finds a solution.

Answer-free recall

Count weights and biases in a $4 \rightarrow 6 \rightarrow 3$ dense network. Explain why two affine layers without activation collapse. State three things universal approximation does not guarantee.

[Know More](#) | [Detailed](#) | [External Sources](#)

Session 3 - Forward pass, backpropagation, and outputs

Intuition

A feedforward network sends values input-to-output without feedback. A computation graph records operations. Backpropagation walks it backward with the chain rule to compute loss gradients; gradient descent uses them.

Definitions and jargon

A logit is a score before probability conversion. A gradient collects loss slopes. The chain rule multiplies local slopes along a path. Sigmoid is $1/(1+\exp(-z))$; tanh is $(\exp(z)-\exp(-z))/(\exp(z)+\exp(-z))$ with range $(-1,1)$; ReLU is $\max(0,z)$. Softmax turns class logits into probabilities; $\exp$ is exponential and $\log$ its inverse.

Formulas, symbols, shapes, and assumptions

For K layers numbered $l=1,\dots,K$, layer l has n_l units, weights W_l , bias b_l , scores z_l , outputs h_l , and activation ϕ_l : $z_l=W_l h_{l-1}+b_l$, $h_l=\phi_l(z_l)$. If W_l is $n_l \times n_{l-1}$, the other three are $n_l \times 1$. Gradient descent is $\theta \leftarrow \theta - \eta dL/d\theta$: L is loss, θ all parameters, $\eta > 0$ the learning rate. Softmax is $p_k = \exp(a_k) / \sum_j \exp(a_j)$ over class logits; subtract the largest logit first.

Conditional properties

For one-hot targets, softmax cross-entropy is $L = -\log(p_y)$. Sigmoid and tanh can saturate and create small derivatives. Backpropagation calculates gradients; the optimizer updates.

Important distinctions

Forward propagation computes predictions/loss; backpropagation computes derivatives; gradient descent changes parameters. Softmax couples exclusive classes; independent labels normally use separate sigmoids.

For an output layer, use a linear output for an unconstrained numerical target, sigmoid for one binary probability, softmax for one mutually exclusive class among many, and independent binary outputs for multiple simultaneous labels. A sign or step activation gives a hard threshold; it is useful for a decision rule but is not a convenient differentiable hidden-layer activation.

When derivatives multiplied through many layers become tiny, gradients vanish; when they become very large, gradients explode. Saturating sigmoid/tanh units can contribute to vanishing gradients, while ReLU has a positive-side gradient but can leave units inactive for negative inputs.

Checked example

Let $x=3$, $z=2x-1=5$, $h=\text{ReLU}(z)=5$, and $L=(h-7)^2$. Then $dL/dx=2(5-7)(1)(2)=-8$. Separately, sigmoid at logit 1 is $1/(1+\exp(-1))=0.731$ (rounded).

Traps and counterexamples

Do not exponentiate huge logits naively. Sum-to-one does not prove calibration. Backpropagation is not an optimizer; cross-entropy is not perceptron error $y-y_{\text{hat}}$.

Answer-free recall

Trace shapes through a 5 → 4 → 3 network. Explain the three stages: forward pass, backward pass, update. Why can subtracting the largest logit leave softmax probabilities unchanged?

[Practice](#) | [Detailed](#) | [External Sources](#)

Session 4 - Optimizing deep models

Intuition

Deep loss is generally nonconvex. Plateaus are flat and slow; saddles curve up in some directions and down in others. Gradients guide updates, but no optimizer guarantees a global minimum.

Definitions and jargon

The gradient is the vector of loss slopes. Batch GD uses all examples, SGD one, and minibatch SGD a subset. Momentum smooths; Nesterov measures at a look-ahead. Rprop uses signs; safeguarded Quickprop uses a local quadratic step. Adagrad, RMSprop, and Adam use past squared gradients.

Formulas, symbols, shapes, and assumptions

At step t , θ_t is the parameter vector and g_t its gradient. Learning rate $\eta > 0$ sets scale; m_t, v_t average gradients and their squares; hats mean bias-corrected. Decays β_1, β_2 lie in $[0, 1)$; $\epsilon > 0$ prevents division by zero. Operations are elementwise.

1 Accumulate moments

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

gradient average; squared-gradient average

2 Correct zero-start bias

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

3 Update each parameter

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

products and divisions are elementwise

Conditional properties

Bias correction fixes Adam's early zero-start bias. Adagrad's accumulator only grows, so steps can shrink. Rprop adapts by sign agreement; Quickprop needs denominator, growth, and sign safeguards.

Important distinctions

SGD chooses samples; momentum and adaptive scaling modify updates. Nesterov evaluates after look-ahead; epsilon stabilizes division, not the learning rate.

Checked example

With Adam $g_1=2$, $m_0=v_0=0$, $\beta_1=0.9$, $\beta_2=0.99$, $\eta=0.03$, ϵ negligible: $m_1=0.2$, $v_1=0.04$, $\hat{m}_1=2$, $\hat{v}_1=4$, so the first update is -0.03 .

Traps and counterexamples

No optimizer always wins. Large η can diverge; tiny η can stall. Quickprop needs caps for near-zero denominators and fallback on sign change.

Answer-free recall

Contrast batch, stochastic, and minibatch gradients. Compute a new first Adam step. Distinguish a saddle, local minimum, and plateau.

[Practice](#) | [Detailed](#) | [External Sources](#)

Session 5 - Generalization and stable training

Intuition

Training loss measures fit to seen examples; the goal is performance on unseen data. Validation supports model choices; an untouched test set estimates final performance.

Definitions and jargon

Underfitting means train and validation results are poor; overfitting means train is strong while validation worsens. L1 adds absolute parameters; L2 their squares. Dropout masks training units. Dataset shift means train/deployment distributions differ. Internal covariate shift is the historical BN claim that layer inputs change as earlier layers learn—not dataset shift.

Formulas, symbols, shapes, and assumptions

$J(\theta) = \text{data_loss} + \lambda R(\theta)$: J is objective, θ parameters, R penalty, $\lambda \geq 0$ strength. L1 uses $\sum_i |\theta_i|$; L2 uses $\sum_i \theta_i^2$, summed over parameter index i . For inverted dropout, h is unmasked activation; mask m keeps entries with probability q , training scales by $1/q$, and inference uses h .

Conditional properties

Initialization can shrink/amplify signals and gradients. Xavier suits some symmetric activations; He scaling suits ReLU families. Gradient clipping caps value or norm to limit explosions, not vanishing. BN uses minibatch statistics in training and stored estimates at inference.

Important distinctions

L1 can encourage zeros; L2 smoothly discourages large weights. Dropout is random only in training. BN's historical motivation does not mean BN solves deployment shift.

Checked example

If unregularized loss is 0.42, $\lambda = 0.05$, and weights are (2, -1, 3), then L1 gives $0.42 + 0.05(6) = 0.72$; L2 gives $0.42 + 0.05(14) = 1.12$. These values compare objectives under the stated penalty definitions, not universal model quality.

Traps and counterexamples

Small training loss can coexist with poor validation. Repeated test use becomes tuning. BN cannot guarantee population robustness; dropout cannot repair bad labels.

Answer-free recall

Diagnose train-low/validation-high loss. Explain train, validation, and test roles. Compare L1, L2, dropout, initialization, and BN by the problem each addresses.

[Practice](#) | [Detailed](#) | [External Sources](#)

Session 6 - Convolutional networks

Intuition

A convolutional layer scans a learned kernel across an image. Sharing that kernel preserves spatial structure and uses fewer parameters than a dense image-wide layer.

Definitions and jargon

This course uses cross-correlation: slide without flipping the kernel, multiply overlaps, and sum. Padding adds borders, stride controls movement, dilation spaces taps, channels hold feature maps, and pooling summarizes regions. Max pooling keeps the largest value; average pooling the mean. Kernels are learned.

Formulas, symbols, shapes, and assumptions

For input size N , padding P , kernel K , stride S , and dilation D , effective kernel $K_{\text{eff}}=D(K-1)+1$ and output size $\text{floor}((N+2P-K_{\text{eff}})/S)+1$; floor rounds down. Apply separately to height/width. With input/output channels $C_{\text{in}}, C_{\text{out}}$, parameters are $K_h \cdot K_w \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$ when each output channel has one bias.

Conditional properties

The formula needs integer settings, $S \geq 1$, $D \geq 1$, and nonnegative numerator. Pooling usually has no learned parameters. Padding preserves size only for compatible kernel, dilation, and stride.

Important distinctions

Parameters depend on kernels/channels, not image height/width. Stride skips positions; dilation spreads taps. Pooling aggregates; convolution learns. Mathematical convolution flips its kernel; cross-correlation does not.

Checked example

Input $15 \times 15 \times 3$, 4 kernels of size 3×3 , padding 2, stride 2, dilation 2: $K_{\text{eff}}=5$; output height and width are $\text{floor}((15+4-5)/2)+1=8$, so shape $8 \times 8 \times 4$. Parameters: $3 \cdot 3 \cdot 3 \cdot 4 + 4 = 112$.

Traps and counterexamples

Do not omit dilation from the output-size formula. “Same” padding is not automatically symmetric for every stride/kernel combination. A larger feature map does not itself increase one layer's parameter count.

Answer-free recall

Compute output shape and parameters for a fresh dilated layer. Explain why parameter sharing is useful. Distinguish padding, stride, dilation, channels, and pooling.

[Practice](#) | [Detailed](#) | [External Sources](#)

Session 7 - CNN design and transfer learning

Intuition

CNNs grow receptive fields and build local patterns into task concepts. Residual connections add a direct identity path. Transfer learning reuses source-task features for applications such as image classification, detection, and medical imaging, subject to domain fit.

Definitions and jargon

The receptive field is the input region influencing one activation. A residual block adds learned change $F(x)$ to x . Freezing fixes reused parameters; fine-tuning updates them. Domain mismatch means source/target differences limit reuse.

Formulas, symbols, shapes, and assumptions

Residual output $y=F(x)+x$ needs matching shapes or a projection. Let r_l be receptive-field size and j_l input spacing between neighboring layer- l positions. With kernel K_l , dilation D_l , stride S_l :
 $r_l=r_{l-1}+D_l(K_l-1)j_{l-1}$ and $j_l=j_{l-1}S_l$.

Conditional properties

Freezing suits little target data or close domains; fine-tuning adapts but costs more and may overfit. Start with a frozen feature extractor and target-specific head, then unfreeze later blocks only if target validation supports it. Pretrained layers often use a small learning rate. AlexNet showed a large trained CNN; ResNet popularized residual learning.

Important distinctions

Focus on feature maps, downsampling, receptive fields, residual paths, and compute—not trivia. Severe domain mismatch can defeat transfer.

Checked example

Two stride-1 3×3 layers start with $r_0=1, j_0=1$. After layer 1, $r_1=3, j_1=1$; after layer 2, $r_2=5$. Thus one final activation can depend on a 5×5 input region.

Traps and counterexamples

Freezing can underfit a new domain; fine-tuning all can overfit tiny data. Residual addition needs matching shapes. Bigger is not automatically better.

Answer-free recall

Compute a receptive field through a stride-2 block. Choose freeze or fine-tune for a small related dataset and justify. Explain what a residual path changes and what it does not guarantee.

Session 8 - Integrated diagnosis and review

Intuition

Session 8 adds no new theory. It asks you to connect symptoms to causes: representation limits, incorrect shapes, unstable optimization, poor generalization, or unsuitable CNN/transfer choices.

Definitions and jargon

A diagnosis links evidence to a likely mechanism and a targeted next check. Representation failure means the model cannot express the rule; optimization failure means training does not find useful parameters; generalization failure means training succeeds but unseen-data performance does not. A validity condition says when a formula applies; one valid counterexample disproves an absolute claim.

Formulas, symbols, shapes, and assumptions

Use a four-line answer frame: (1) name the governing idea or formula; (2) substitute values or trace shapes; (3) state conditions; (4) interpret and propose one targeted action. Keep the key formulas ready: perceptron update, dense parameter count, chain-rule product, Adam bias correction, regularized objective, and dilated CNN output size.

Conditional properties

The same symptom can have several causes. Flat training may reflect saturation, a plateau, unsuitable learning rate, or data problems; distinguish them with gradients, activation ranges, and train/validation curves. A recommendation is conditional on the evidence supplied.

Important distinctions

Training failure differs from generalization failure. A wrong shape differs from a wrong numerical value. Underfitting differs from optimization not yet converged. Dataset shift differs from overfitting, though both can hurt deployment performance.

When comparing two architectures, do not choose by neuron count alone. Check target/output compatibility, ability to represent the task, training stability, validation performance, computation, and risk of overfitting. More capacity helps only when the data and training evidence support it.

Checked example

A model has training loss falling from 0.80 to 0.08, validation loss bottoming at 0.31 then rising to 0.57. This is evidence of overfitting after the validation minimum, not vanishing gradients. A targeted response is early stopping or stronger regularization, followed by validation; the untouched test set remains for final evaluation.

Traps and counterexamples

Do not recommend Adam for every problem, BN for every shift, more depth for every underfit, or transfer learning regardless of source-target mismatch. Do not diagnose from one metric without its split and trend.

Answer-free recall

Create one diagnosis each for XOR failure, exploding activations, widening train-validation gap, invalid residual addition, and negative CNN output numerator. For each, state one check and one bounded action.

[Practice](#) | [Detailed](#) | [External Sources](#)